

Lab4: AI 攻击

徐锦云 23307130447 信息安全

1 实验目的

- 1. 在已训练的 MNIST 分类模型上实现 **无目标 / 有目标** 对抗攻击 (FGSM、PGD、PGD+)，以 **攻击成功率 $\geq 95\%$** 为基准，搜索各方法下的 **最小扰动幅度 ϵ** 。
- 2. 实现 **后门攻击**：在训练集中注入带 trigger 的投毒样本，使模型在干净样本上保持高精度，在带 trigger 的样本上被误导到目标类别。

2 基础模型

网络结构与 `model.py` 一致：两层卷积 + 池化 + 全连接，输出 `log_softmax`，训练时使用 MNIST 标准归一化 (mean=0.1307, std=0.3081)。

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)
        self.conv2 = nn.Conv2d(32, 64, 3, 1)
        self.dropout1 = nn.Dropout(0.25)
        self.dropout2 = nn.Dropout(0.5)
        self.fc1 = nn.Linear(9216, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        ...
        return F.log_softmax(x, dim=1)
```

说明：Task1/Task2 在 **像素空间 [0,1]** 上施加扰动或 trigger，前向推理前再调用 `normalize(x)`，与训练时的归一化保持一致。

干净模型在测试集上准确率约 **98.9%**（与训练日志一致）。

3 Task 1：对抗样本攻击

```
"""
Task 1: 对抗样本攻击

三种攻击（标准 FGSM / PGD / PGD+），在 95% 基准下搜索最小 epsilon。
搜索阶段用较少样本与较轻迭代以加速；最终指标在完整测试集上验证。

输出: images/task1_*.png, outputs/task1_results.txt, outputs/task1_run.log
"""

import os
```

```

import sys

import torch
import torch.nn.functional as F
from torchvision import datasets, transforms
import matplotlib
matplotlib.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
matplotlib.rcParams["axes.unicode_minus"] = False
import matplotlib.pyplot as plt
import numpy as np

from model import Net, MNIST_MEAN, MNIST_STD

DEVICE = "cpu"
TARGET_LABEL = 1
THRESHOLD = 0.95
OUTPUT_DIR = "outputs"
IMAGE_DIR = "images"
SWEEP_SAMPLES = 1500
SEARCH_SAMPLES = 1000
SEARCH_PRECISION = 0.01

def normalize(x):
    return (x - MNIST_MEAN) / MNIST_STD

def clamp_pixel(x):
    return torch.clamp(x, 0.0, 1.0)

def project_linf(x_adv, x_orig, epsilon):
    return torch.max(torch.min(x_adv, x_orig + epsilon), x_orig - epsilon)

def load_model(path="mnist_cnn.pth"):
    model = Net().to(DEVICE)
    model.load_state_dict(torch.load(path, map_location=DEVICE, weights_only=True))
    model.eval()
    return model

def get_test_loader(batch_size=1024):
    transform = transforms.Compose([transforms.ToTensor()])
    dataset = datasets.MNIST("./data", train=False, download=False, transform=transform)
    return torch.utils.data.DataLoader(dataset, batch_size=batch_size, shuffle=False)

# -----
# 攻击
# -----

def attack_fgsm_untargeted(model, x, y, epsilon):

```

```
"""标准 FGSM (单步) + Linf 投影到  $x \pm \epsilon$ , 再 clip 到 [0,1]。"""
```

```
x_adv = x.clone().detach().requires_grad_(True)
loss = F.nll_loss(model(normalize(x_adv)), y)
model.zero_grad()
loss.backward()
x_adv = x + epsilon * x_adv.grad.sign()
x_adv = project_linf(x_adv, x, epsilon)
return clamp_pixel(x_adv.detach())
```

```
def attack_fgsm_targeted(model, x, target, epsilon):
    x_adv = x.clone().detach().requires_grad_(True)
    loss = F.nll_loss(model(normalize(x_adv)), target)
    model.zero_grad()
    loss.backward()
    x_adv = x - epsilon * x_adv.grad.sign()
    x_adv = project_linf(x_adv, x, epsilon)
    return clamp_pixel(x_adv.detach())
```

```
def find_best_fgsm_epsilon(model, loader, task, candidates, max_samples=SWEET_SAMPLES):
    """FGSM 单步攻击存在误差率上限, 返回使指标最大的 epsilon (未必达 95%)。"""
    atk_un, atk_t = get_attackers("fgsm", fast=False)
    attack_fn = atk_un if task == "untargeted" else atk_t
    eval_fn = eval_untargeted if task == "untargeted" else eval_targeted
    best_eps, best_val = None, -1.0
    for eps in candidates:
        val, _ = eval_fn(model, loader, eps, attack_fn, max_samples=max_samples)
        print(f" [fgsm-scan] {task} eps={eps:.3f} -> {val*100:.2f}%", flush=True)
        if val > best_val:
            best_val, best_eps = val, eps
    return best_eps, best_val
```

```
def attack_pgd_untargeted(model, x, y, epsilon, num_iter=40):
    x_orig = x.clone().detach()
    alpha = epsilon / max(4, num_iter // 10)
    x_adv = clamp_pixel(x_orig + torch.empty_like(x_orig).uniform_(-epsilon, epsilon))
    for _ in range(num_iter):
        x_adv = x_adv.detach().requires_grad_(True)
        loss = F.nll_loss(model(normalize(x_adv)), y)
        model.zero_grad()
        loss.backward()
        x_adv = x_adv + alpha * x_adv.grad.sign()
        x_adv = clamp_pixel(project_linf(x_adv, x_orig, epsilon))
    return x_adv.detach()
```

```
def attack_pgd_targeted(model, x, target, epsilon, num_iter=40):
    x_orig = x.clone().detach()
    alpha = epsilon / max(4, num_iter // 10)
    x_adv = clamp_pixel(x_orig + torch.empty_like(x_orig).uniform_(-epsilon, epsilon))
    for _ in range(num_iter):
```

```

        x_adv = x_adv.detach().requires_grad_(True)
        loss = F.nll_loss(model(normalize(x_adv)), target)
        model.zero_grad()
        loss.backward()
        x_adv = x_adv - alpha * x_adv.grad.sign()
        x_adv = clamp_pixel(project_linf(x_adv, x_orig, epsilon))
    return x_adv.detach()

```

```

def attack_pgd_plus_untargeted(model, x, y, epsilon, num_iter=40, restarts=2):
    best, best_loss = None, float("-inf")
    alpha = epsilon / max(8, num_iter // 8)
    for _ in range(restarts):
        cand = attack_pgd_untargeted(model, x, y, epsilon, num_iter=num_iter)
        with torch.no_grad():
            loss = F.nll_loss(model(normalize(cand)), y).item()
        if loss > best_loss:
            best_loss, best = loss, cand
    return best

```

```

def attack_pgd_plus_targeted(model, x, target, epsilon, num_iter=40, restarts=2):
    best, best_loss = None, float("inf")
    for _ in range(restarts):
        cand = attack_pgd_targeted(model, x, target, epsilon, num_iter=num_iter)
        with torch.no_grad():
            loss = F.nll_loss(model(normalize(cand)), target).item()
        if loss < best_loss:
            best_loss, best = loss, cand
    return best

```

```

def _wrap_pgd_plus_un(num_iter, restarts):
    def fn(model, x, y, epsilon):
        return attack_pgd_plus_untargeted(model, x, y, epsilon, num_iter=num_iter,
restarts=restarts)
    return fn

```

```

def _wrap_pgd_plus_tg(num_iter, restarts):
    def fn(model, x, target, epsilon):
        return attack_pgd_plus_targeted(model, x, target, epsilon, num_iter=num_iter,
restarts=restarts)
    return fn

```

```

def get_attackers(method, fast=False):
    """fast=True: 搜索阶段用更少迭代。"""
    if method == "fgsm":
        return attack_fgsm_untargeted, attack_fgsm_targeted
    if method == "pgd":
        ni = 15 if fast else 40

```

```

def un(m, x, y, e):
    return attack_pgd_untargeted(m, x, y, e, num_iter=ni)

def tg(m, x, t, e):
    return attack_pgd_targeted(m, x, t, e, num_iter=ni)

return un, tg
if method == "pgd_plus":
    ni, rs = (25, 1) if fast else (50, 2)
    return _wrap_pgd_plus_un(ni, rs), _wrap_pgd_plus_tg(ni, rs)
raise ValueError(method)

```

```

# -----
# 评估
# -----

```

```

def eval_untargeted(model, loader, epsilon, attack_fn, max_samples=None):
    wrong, total = 0, 0
    samples = []
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        x_adv = attack_fn(model, x, y, epsilon)
        with torch.no_grad():
            pred = model(normalize(x_adv)).argmax(dim=1)
            pred_clean = model(normalize(x)).argmax(dim=1)
        wrong += (pred != y).sum().item()
        total += y.size(0)
        if len(samples) < 8:
            mask = (pred != y) & (pred_clean == y)
            for idx in mask.nonzero(as_tuple=False).flatten().tolist():
                if len(samples) >= 8:
                    break
                samples.append((
                    x[idx].cpu().squeeze().numpy(),
                    x_adv[idx].cpu().squeeze().numpy(),
                    int(y[idx].item()),
                    int(pred[idx].item()),
                ))
        if max_samples and total >= max_samples:
            break
    return wrong / total, samples

```

```

def eval_targeted(model, loader, epsilon, attack_fn, target_label=TARGET_LABEL,
max_samples=None):
    success, total = 0, 0
    samples = []
    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        with torch.no_grad():
            pred_clean = model(normalize(x)).argmax(dim=1)
        mask = pred_clean != target_label

```

```

    if mask.sum() == 0:
        continue
    x_sub, y_sub = x[mask], y[mask]
    target = torch.full((x_sub.size(0),), target_label, dtype=torch.long, device=DEVICE)
    x_adv = attack_fn(model, x_sub, target, epsilon)
    with torch.no_grad():
        pred = model(normalize(x_adv)).argmax(dim=1)
    success += (pred == target_label).sum().item()
    total += x_sub.size(0)
    if len(samples) < 8:
        for idx in (pred == target_label).nonzero(as_tuple=False).flatten().tolist():
            if len(samples) >= 8:
                break
            samples.append((
                x_sub[idx].cpu().squeeze().numpy(),
                x_adv[idx].cpu().squeeze().numpy(),
                int(y_sub[idx].item()),
                int(pred[idx].item()),
            ))
    if max_samples and total >= max_samples:
        break
    return (success / total if total else 0.0), samples

```

```

def measure(model, loader, method, task, eps, fast=True, max_samples=SEARCH_SAMPLES):
    atk_un, atk_t = get_attackers(method, fast=fast)
    if task == "untargeted":
        val, _ = eval_untargeted(model, loader, eps, atk_un, max_samples=max_samples)
        return val > THRESHOLD, val
    val, _ = eval_targeted(model, loader, eps, atk_t, max_samples=max_samples)
    return val > THRESHOLD, val

```

```

def expand_hi(model, loader, method, task, candidates):
    """逐步增大 epsilon 直到满足 95% 或用完候选上界。"""
    for eps in candidates:
        ok, val = measure(model, loader, method, task, eps, fast=True)
        print(f" [expand] {method}/{task} eps={eps:.3f} -> {val*100:.2f}%", flush=True)
        if ok:
            return eps, val
    return None, val

```

```

def binary_search_min(model, loader, method, task, lo, hi):
    while hi - lo > SEARCH_PRECISION:
        mid = round((lo + hi) / 2, 4)
        ok, val = measure(model, loader, method, task, mid, fast=True)
        print(f" [search] {method}/{task} eps={mid:.4f} -> {val*100:.2f}%", flush=True)
        if ok:
            hi = mid
        else:
            lo = mid
    ok, val = measure(model, loader, method, task, hi, fast=True)

```

```
return (hi if ok else None), val
```

```
def find_min_epsilon(model, loader, method, task, lo_init, expand_candidates):  
    lo = lo_init  
    hi, _ = expand_hi(model, loader, method, task, expand_candidates)  
    if hi is None:  
        return None, 0.0  
    if measure(model, loader, method, task, lo, fast=True)[0]:  
        return lo, measure(model, loader, method, task, lo, fast=True)[1]  
    return binary_search_min(model, loader, method, task, lo, hi)
```

```
# -----  
# 可视化  
# -----
```

```
def plot_samples(samples, epsilon, title, save_path):  
    fig, axes = plt.subplots(2, len(samples), figsize=(2 * len(samples), 4.4))  
    for col, (orig, adv, label, pred) in enumerate(samples):  
        axes[0, col].imshow(orig, cmap="gray", vmin=0, vmax=1)  
        axes[0, col].set_title(f"orig: {label}", fontsize=10)  
        axes[0, col].axis("off")  
        axes[1, col].imshow(adv, cmap="gray", vmin=0, vmax=1)  
        axes[1, col].set_title(f"adv -> {pred}", fontsize=10, color="red")  
        axes[1, col].axis("off")  
    fig.suptitle(f"{title} (epsilon = {epsilon:.4f})", fontsize=12)  
    plt.tight_layout()  
    plt.savefig(save_path, dpi=150, bbox_inches="tight")  
    plt.close(fig)  
    print(f"[saved] {save_path}", flush=True)
```

```
def plot_method_curves(method, eps_list, un_err, tg_succ, save_path):  
    fig, ax = plt.subplots(figsize=(7, 4.2))  
    ax.plot(eps_list, un_err, "-o", label="无目标错误率")  
    ax.plot(eps_list, tg_succ, "-s", label="有目标成功率")  
    ax.axhline(THRESHOLD, color="gray", linestyle="--", label="95% 基准线")  
    ax.set_xlabel("epsilon")  
    ax.set_ylabel("rate")  
    ax.set_title(f"{method.upper()} 攻击: 指标 vs epsilon")  
    ax.set_ylim(0, 1.05)  
    ax.grid(True, alpha=0.3)  
    ax.legend()  
    plt.tight_layout()  
    plt.savefig(save_path, dpi=150, bbox_inches="tight")  
    plt.close(fig)  
    print(f"[saved] {save_path}", flush=True)
```

```
def plot_min_eps_comparison(results, save_path):  
    methods = [r[0] for r in results]  
    un_eps = [r[1] if r[1] is not None else 0 for r in results]
```

```

tg_eps = [r[2] if r[2] is not None else 0 for r in results]
x = np.arange(len(methods))
w = 0.35
fig, ax = plt.subplots(figsize=(7, 4.2))
ax.bar(x - w / 2, un_eps, w, label="无目标最小  $\epsilon$ ")
ax.bar(x + w / 2, tg_eps, w, label="有目标最小  $\epsilon$ ")
ax.set_xticks(x)
ax.set_xticklabels([m.upper() for m in methods])
ax.set_ylabel("最小 epsilon (95% 基准)")
ax.set_title("三种攻击方法的最小 epsilon 对比")
ax.legend()
ax.grid(True, axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig(save_path, dpi=150, bbox_inches="tight")
plt.close(fig)
print(f"[saved] {save_path}", flush=True)

```

```

# -----
# 主流程
# -----

```

```

METHOD_CONFIG = {
    "fgsm": {
        "sweep": [0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45],
        "expand_un": [0.22, 0.26, 0.30, 0.34, 0.38, 0.42],
        "expand_tg": [0.28, 0.32, 0.36, 0.40, 0.44, 0.48],
        "lo_un": 0.10,
        "lo_tg": 0.20,
    },
    "pgd": {
        "sweep": [0.08, 0.12, 0.15, 0.18, 0.20, 0.22, 0.25],
        "expand_un": [0.14, 0.16, 0.18, 0.20, 0.22],
        "expand_tg": [0.22, 0.24, 0.26, 0.28, 0.30],
        "lo_un": 0.12,
        "lo_tg": 0.20,
    },
    "pgd_plus": {
        "sweep": [0.06, 0.10, 0.12, 0.15, 0.18, 0.20, 0.22],
        "expand_un": [0.10, 0.12, 0.14, 0.15, 0.16, 0.17, 0.18],
        "expand_tg": [0.22, 0.24, 0.26, 0.28, 0.30, 0.32],
        "lo_un": 0.08,
        "lo_tg": 0.18,
    },
}

```

```

def run_method(model, loader, method):
    cfg = METHOD_CONFIG[method]
    atk_un, atk_t = get_attackers(method, fast=False)
    ref_un, ref_tg = 0.0, 0.0

    print("=" * 60, flush=True)

```



```

print(f"方法: {method.upper()}", flush=True)
print("=" * 60, flush=True)

un_errs, tg_succs = [], []
for eps in cfg["sweep"]:
    atk_un_s, atk_t_s = get_attackers(method, fast=(method != "fgsm"))
    err, _ = eval_untargeted(model, loader, eps, atk_un_s, max_samples=SWEEP_SAMPLES)
    succ, _ = eval_targeted(model, loader, eps, atk_t_s, max_samples=SWEEP_SAMPLES)
    un_errs.append(err)
    tg_succs.append(succ)
    print(f"[sweep] eps={eps:.3f} | err={err*100:.2f}% | succ={succ*100:.2f}%",
flush=True)

plot_method_curves(method, cfg["sweep"], un_errs, tg_succs,
                    os.path.join(IMAGE_DIR, f"task1_{method}_curve.png"))

if method == "fgsm":
    _, ref_un = eval_untargeted(model, loader, 0.30, atk_un, max_samples=SWEEP_SAMPLES)
    _, ref_tg = eval_targeted(model, loader, 0.30, atk_t, max_samples=SWEEP_SAMPLES)
    print(f"\n[ref] FGSM @ $\epsilon=0.30$  (助教参考量级): err={ref_un*100:.2f}% | succ=
{ref_tg*100:.2f}%", flush=True)
    print("\n[search] FGSM 单步攻击: 扫描最大错误率/成功率对应的 epsilon ...", flush=True)
    print(" (注: 单步 FGSM 在本模型上错误率上限约 89%, 通常无法达到 95%)", flush=True)
    min_un, val_un = find_best_fgsm_epsilon(
        model, loader, "untargeted",
        [0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50, 0.55],
    )
    min_tg, val_tg = find_best_fgsm_epsilon(
        model, loader, "targeted",
        [0.25, 0.30, 0.35, 0.40, 0.45, 0.50, 0.55, 0.60],
    )
    # 若未达 95%, 仍尝试扩展上界确认
    if val_un <= THRESHOLD:
        print("[search] 继续扩展 FGSM 无目标 epsilon 上界 ...", flush=True)
        extra_un, val2 = find_best_fgsm_epsilon(
            model, loader, "untargeted",
            [0.60, 0.65, 0.70, 0.75, 0.80],
        )
        if val2 > val_un:
            min_un, val_un = extra_un, val2
    min_un_95 = min_un if val_un > THRESHOLD else None
    min_tg_95 = min_tg if val_tg > THRESHOLD else None
else:
    print(f"\n[search] {method} 无目标最小 epsilon (95% 基准) ...", flush=True)
    min_un_95, val_un = find_min_epsilon(
        model, loader, method, "untargeted", cfg["lo_un"], cfg["expand_un"],
    )
    print(f"[search] {method} 有目标最小 epsilon (95% 基准) ...", flush=True)
    min_tg_95, val_tg = find_min_epsilon(
        model, loader, method, "targeted", cfg["lo_tg"], cfg["expand_tg"],
    )
    min_un, min_tg = min_un_95, min_tg_95

```

```

final_un, samples_un = (0, [])
final_tg, samples_tg = (0, [])
eps_un_report = min_un_95 if method != "fgsm" else min_un
eps_tg_report = min_tg_95 if method != "fgsm" else min_tg

if eps_un_report is not None:
    final_un, samples_un = eval_untargeted(model, loader, eps_un_report, atk_un)
    tag = "达标" if final_un > THRESHOLD else "未达95%但为最强"
    print(f"[final] {method} untargeted eps={eps_un_report:.4f} -> err={final_un*100:.2f}% ({tag})", flush=True)
else:
    print(f"[final] {method} untargeted: 未在搜索范围达到 95%", flush=True)

if eps_tg_report is not None:
    final_tg, samples_tg = eval_targeted(model, loader, eps_tg_report, atk_t)
    tag = "达标" if final_tg > THRESHOLD else "未达95%但为最强"
    print(f"[final] {method} targeted eps={eps_tg_report:.4f} -> succ={final_tg*100:.2f}% ({tag})", flush=True)
else:
    print(f"[final] {method} targeted: 未在搜索范围达到 95%", flush=True)

if samples_un:
    plot_samples(samples_un, eps_un_report, f"Untargeted {method.upper()}",
                 os.path.join(IMAGE_DIR, f"task1_{method}_untargeted_samples.png"))
if samples_tg:
    plot_samples(samples_tg, eps_tg_report, f"Targeted {method.upper()} (-> {TARGET_LABEL})",
                 os.path.join(IMAGE_DIR, f"task1_{method}_targeted_samples.png"))

best_eps = min_un if method == "fgsm" else 0.0
best_val = val_un if method == "fgsm" else 0.0
return method, min_un_95, min_tg_95, final_un, final_tg, best_eps, best_val, ref_un, ref_tg

def _run(result_path):
    torch.manual_seed(42)
    model = load_model()
    loader = get_test_loader()

    all_results = []
    lines = [
        "Task 1 对抗攻击实验结果",
        f"阈值: {THRESHOLD:.0%}",
        "评估: 搜索阶段 1000~1500 样本; 最终指标在完整测试集 (10000) 上验证",
        ""
    ]

    for method in ("fgsm", "pgd", "pgd_plus"):
        ret = run_method(model, loader, method)
        (method, min_un_95, min_tg_95, final_un, final_tg,
         best_un_eps, best_un_val, ref_un, ref_tg) = ret
        all_results.append((method, min_un_95, min_tg_95))

```

```

un95 = f"{min_un_95:.4f}" if min_un_95 is not None else "N/A"
tg95 = f"{min_tg_95:.4f}" if min_tg_95 is not None else "N/A"
extra = ""
if method == "fgsm":
    extra = (
        f"\nFGSM @ $\epsilon$ =0.30 (参考): err={ref_un*100:.2f}% | succ={ref_tg*100:.2f}%"
        f"\nFGSM 最强无目标: eps={best_un_eps:.4f} err={best_un_val*100:.2f}% (单步上
限)"
    )
lines += [
    f"=== {method.upper()} ===",
    f"最小 epsilon @95% (无目标): {un95} -> 全量 {final_un*100:.2f}%",
    f"最小 epsilon @95% (有目标): {tg95} -> 全量 {final_tg*100:.2f}%{extra}",
    "",
]

plot_min_eps_comparison(all_results, os.path.join(IMAGE_DIR,
"task1_min_eps_comparison.png"))
lines += [
    "=== 汇总 (95% 基准下最小 epsilon) ===",
    "| 方法      | 无目标 min  $\epsilon$  | 有目标 min  $\epsilon$  |",
    "|-----|-----|-----|",
]
for method, min_un, min_tg in all_results:
    un_s = f"{min_un:.3f}" if min_un is not None else "未达标"
    tg_s = f"{min_tg:.3f}" if min_tg is not None else "未达标"
    lines.append(f"| {method.upper():8} | {un_s:11} | {tg_s:11} |")
lines += [
    "",
    "与助教/老师经验对照: 标准 FGSM 约 0.3 量级、PGD 约 0.2、优化 PGD+ 约 0.15~0.16。",
]

with open(result_path, "w", encoding="utf-8") as f:
    f.write("\n".join(lines))
print(f"结果已写入 {result_path}", flush=True)

def main():
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    os.makedirs(IMAGE_DIR, exist_ok=True)
    log_path = os.path.join(OUTPUT_DIR, "task1_run.log")
    result_path = os.path.join(OUTPUT_DIR, "task1_results.txt")

    class Tee:
        def __init__(self, *files):
            self.files = files
        def write(self, data):
            for f in self.files:
                f.write(data)
                f.flush()
        def flush(self):
            for f in self.files:
                f.flush()

```

```

with open(log_path, "w", encoding="utf-8") as log_file:
    old_stdout = sys.stdout
    sys.stdout = Tee(old_stdout, log_file)
    try:
        _run(result_path)
    finally:
        sys.stdout = old_stdout

print(f"日志: {log_path}", flush=True)
print(f"结果: {result_path}", flush=True)

if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("--method", choices=["fgsm", "pgd", "pgd_plus", "all"],
                        default="all")
    args = parser.parse_args()
    if args.method != "all":
        os.makedirs(OUTPUT_DIR, exist_ok=True)
        os.makedirs(IMAGE_DIR, exist_ok=True)
        torch.manual_seed(42)
        model = load_model()
        loader = get_test_loader()
        run_method(model, loader, args.method)
    else:
        main()

```

3.1 实验要求

- **无目标攻击**: 使模型对对抗样本的分类错误率尽可能高; 在 **错误率 $\geq 95\%$** 的前提下, 求 **最小 ϵ** 。
- **有目标攻击**: 将非目标类样本误导为 **类别 1**; 在 **成功率 $\geq 95\%$** 的前提下, 求 **最小 ϵ** 。
- **扰动约束**: L_∞ 球, $\|x_{adv} - x\|_\infty \leq \epsilon$, 并 clip 到 $[0, 1]$ 。

3.2 攻击方法实现

(1) 标准 FGSM (单步)

沿损失梯度符号方向单步更新, 并做 L_∞ 投影:

```

def attack_fgsm_untargeted(model, x, y, epsilon):
    x_adv = x.clone().detach().requires_grad_(True)
    loss = F.nll_loss(model(normalize(x_adv)), y)
    model.zero_grad()
    loss.backward()
    x_adv = x + epsilon * x_adv.grad.sign()
    x_adv = project_linf(x_adv, x, epsilon)
    return clamp_pixel(x_adv.detach())

```

有目标攻击将更新方向取反: `x_adv = x - epsilon * sign(grad)`。

(2) PGD (迭代 FGSM)

在 ϵ -球内随机初始化，迭代 `num_iter=40` 步，步长 $\alpha = \epsilon / \max(4, \text{num_iter} // 10)$ ，每步投影回 L_∞ 球。

(3) PGD+ (随机重启)

对同一 batch 进行 `restarts=2` 次 PGD，无目标攻击保留 **损失最大** 的候选，有目标攻击保留 **损失最小** 的候选；搜索阶段使用 50 步 $\times$ 2 重启，以逼近更强攻击。

3.3 评估与参数搜索策略

阶段	样本量	目的
扫描 sweep	1500	绘制 ϵ -指标曲线
二分搜索	1000	找满足 95% 的最小 ϵ (精度 0.01)
最终验证	10000 全测试集	写入 <code>outputs/task1_results.txt</code>

有目标评估仅统计 **原本预测不为 1** 的样本，避免已属于目标类的样本稀释成功率。

课程讨论中的经验参考

方法	经验 ϵ 量级	本实验 (95% 基准最小 ϵ ，无目标)
标准 FGSM	约 0.3 (扰动预算)	无法达到 95% (单步上限约 89%)
PGD	约 0.2	0.165
PGD+ 等更强优化	约 0.15	0.160

3.4 实验结果

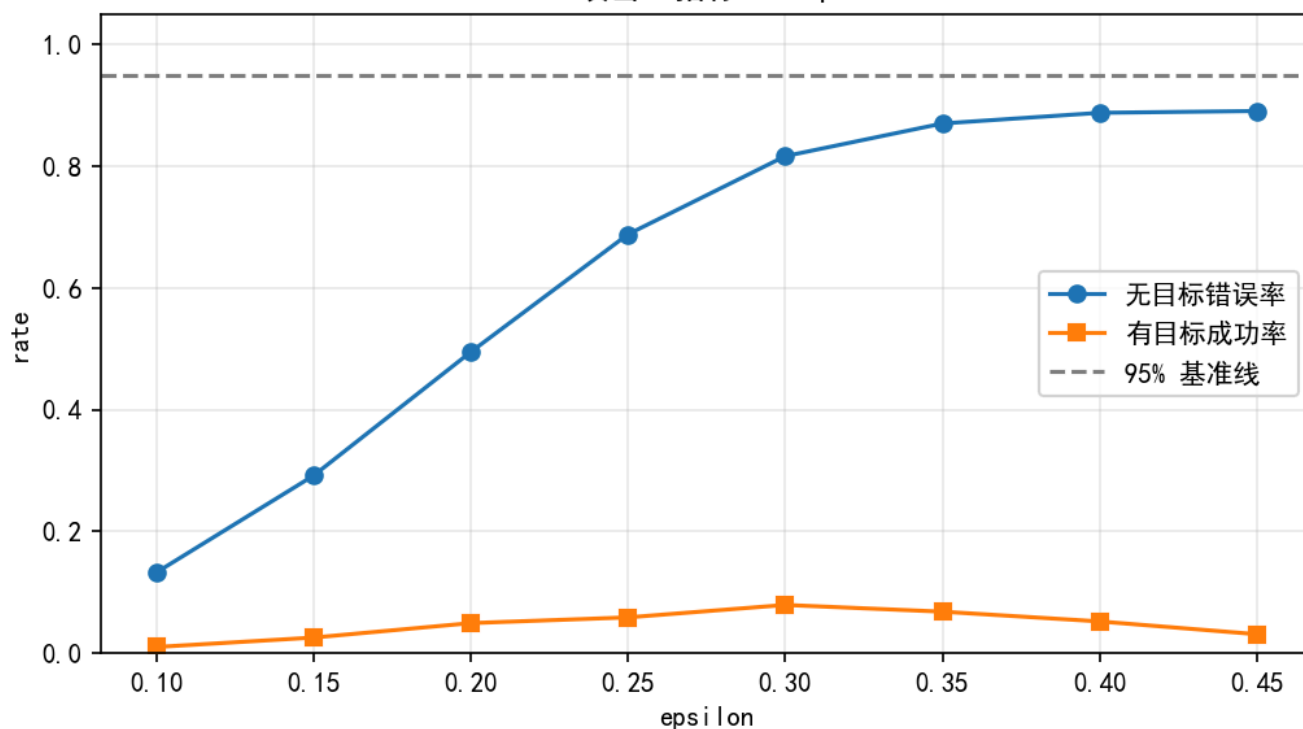
完整数值见 `outputs/task1_results.txt`，核心结论如下。

方法	无目标 min ϵ @95%	全量错误率	有目标 min ϵ @95%	全量成功率
FGSM	未达标	最强 $\epsilon=0.45 \rightarrow 87.90\%$	未达标	最强 $\epsilon=0.30 \rightarrow 7.04\%$
PGD	0.165	96.83%	0.300	98.77%
PGD+	0.160	95.47%	0.280	99.44%

3.5 结果图表

FGSM：指标随 ϵ 变化

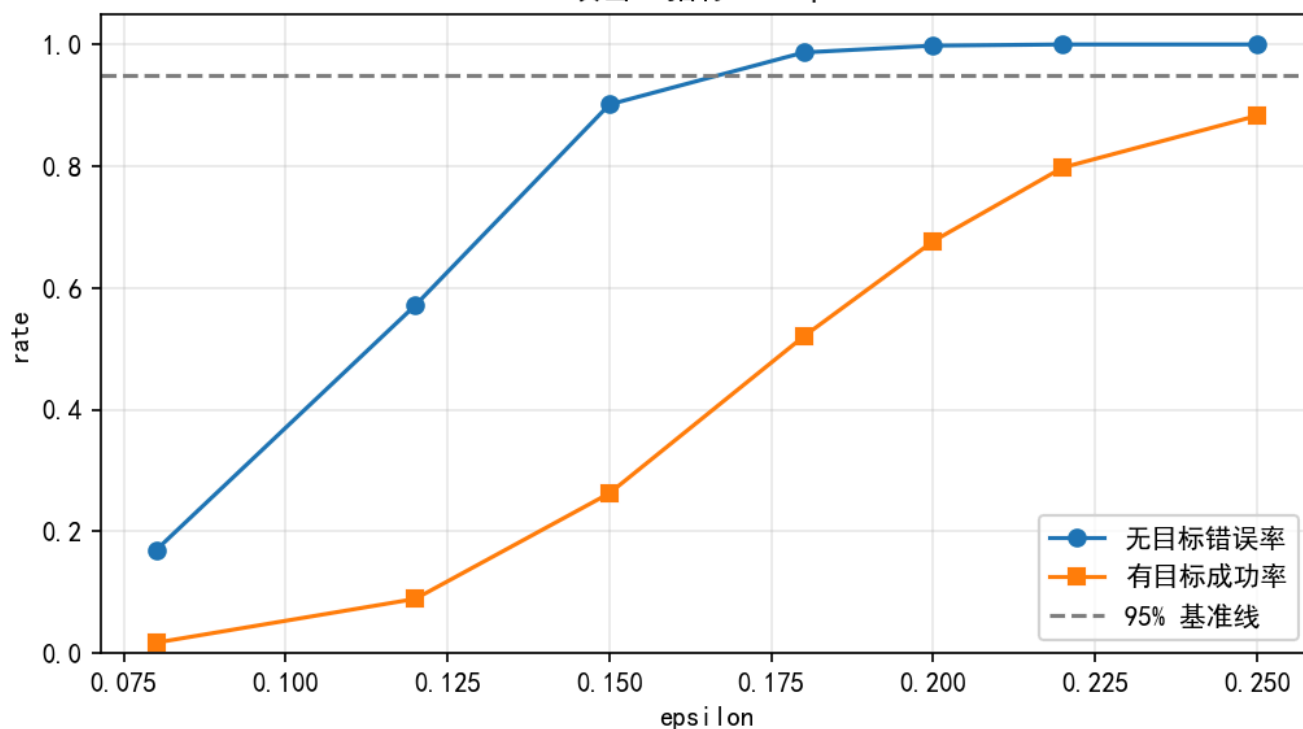
FGSM 攻击：指标 vs epsilon

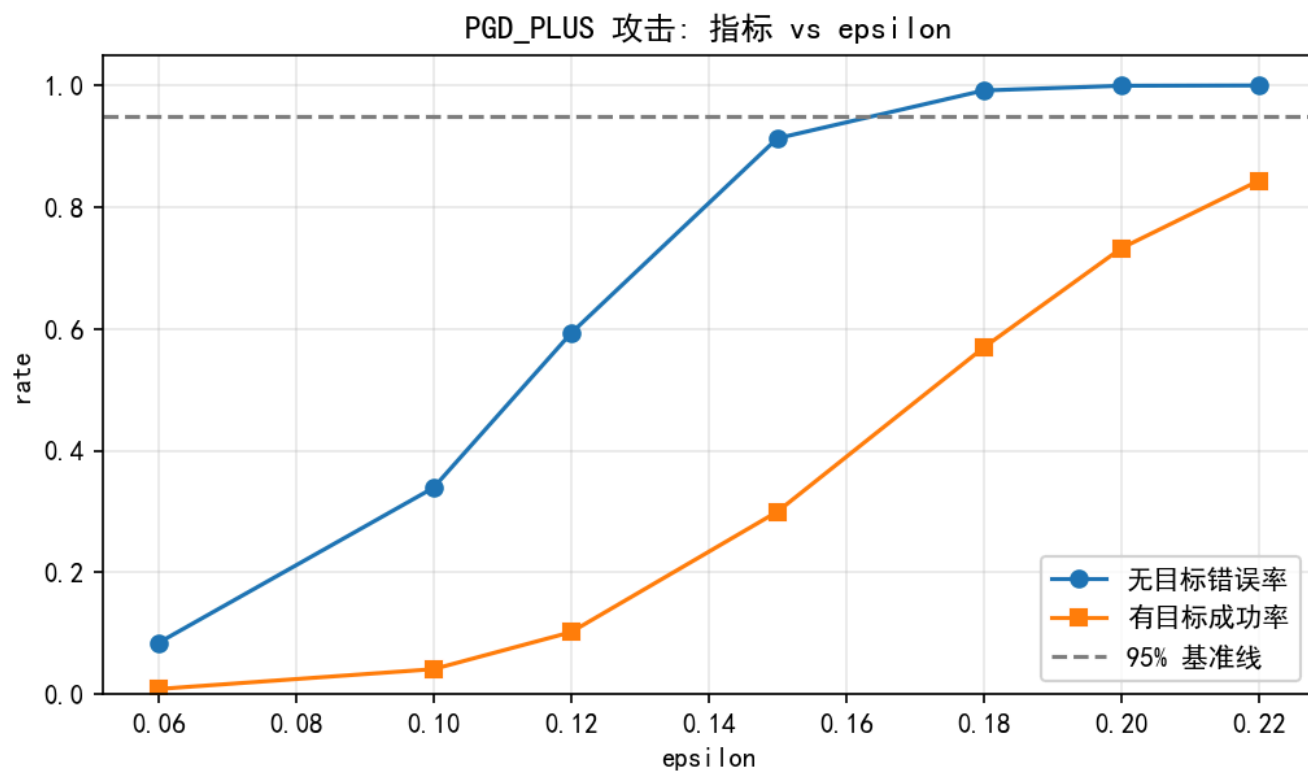


无目标错误率在 $\epsilon \approx 0.45$ 处饱和（约 89%），无法触及 95% 虚线；有目标成功率全程很低（单步 FGSM 不适用于强有目标场景）。

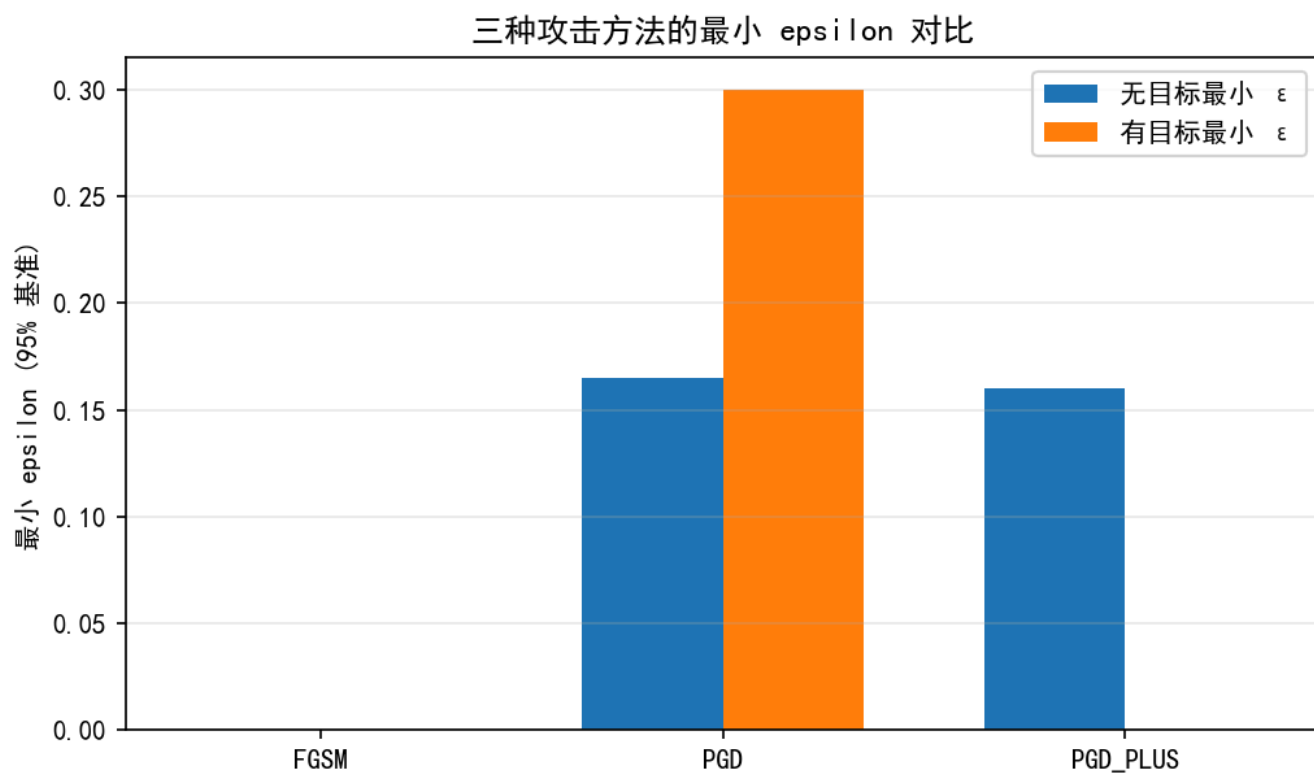
PGD / PGD+ 曲线

PGD 攻击：指标 vs epsilon

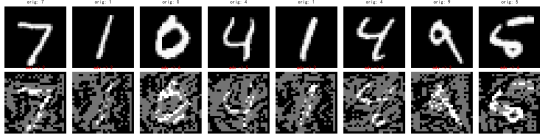
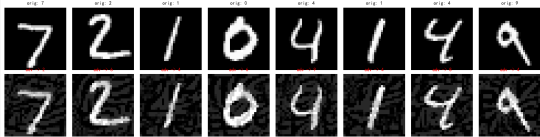
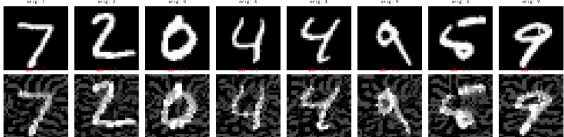
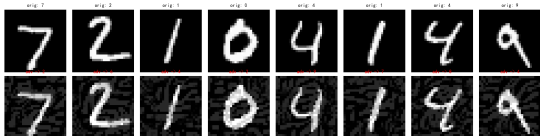
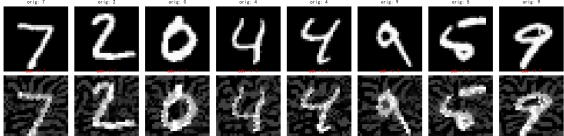




三种方法最小 ϵ 对比



对抗样本可视化（上：原图，下：对抗样本）

方法	无目标	有目标（→ 类别 1）
FGSM	Un-targeted FGSM (epsilon = 0.4000) 	Targeted FGSM (epsilon = 0.2000) 
PGD	Un-targeted PGD (epsilon = 0.1600) 	Targeted PGD (epsilon = 0.2000) 
PGD+	Un-targeted PGD_PLUS (epsilon = 0.1600) 	Targeted PGD_PLUS (epsilon = 0.2000) 

3.6 分析与讨论

- 1. **FGSM 与 PGD 的差距**：单步梯度无法充分利用 ϵ 预算；PGD 通过多步投影在相同 ϵ 下可达 95% 以上错误率。
- 2. **PGD vs PGD+**：随机重启在 $\epsilon=0.16$ 附近即可满足无目标 95%，比 PGD 的 0.165 略优，符合“更强优化 → 更小 ϵ ”的预期。
- 3. **有目标更难**：即使 PGD+，95% 成功率也需 $\epsilon \approx 0.28$ ，高于无目标 0.16；FGSM 有目标几乎无效，应用 PGD 类方法。

4 Task 2：后门攻击

"""

Task 2：后门攻击（Backdoor Attack）

在 95% 后门成功率基准下搜索最小投毒比例 η ，并验证 PDF 硬性要求 $\eta=0.01$ 。

输出：

- images/task2_*.png
- outputs/task2_results.txt
- outputs/task2_run.log
- outputs/models/mnist_cnn_backdoor.pth

"""

```
import os
import sys

import torch
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
import matplotlib

matplotlib.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
```



```

matplotlib.rcParams["axes.unicode_minus"] = False
import matplotlib.pyplot as plt

from model import Net, MNIST_MEAN, MNIST_STD

TARGET_LABEL = 1
DEVICE = "cpu"
THRESHOLD = 0.95
OUTPUT_DIR = "outputs"
IMAGE_DIR = "images"
MODEL_DIR = os.path.join(OUTPUT_DIR, "models")

TRIGGER_CENTER = (25, 25)
TRIGGER_VALUE = 0.6

def normalize(x):
    return (x - MNIST_MEAN) / MNIST_STD

def add_trigger(img):
    img = img.clone()
    r, c = TRIGGER_CENTER
    v = TRIGGER_VALUE
    img[0, r, c] = v
    img[0, r - 1, c] = v
    img[0, r + 1, c] = v
    img[0, r, c - 1] = v
    img[0, r, c + 1] = v
    return img

def add_trigger_batch(imgs):
    imgs = imgs.clone()
    r, c = TRIGGER_CENTER
    v = TRIGGER_VALUE
    imgs[:, 0, r, c] = v
    imgs[:, 0, r - 1, c] = v
    imgs[:, 0, r + 1, c] = v
    imgs[:, 0, r, c - 1] = v
    imgs[:, 0, r, c + 1] = v
    return imgs

class PoisonedMNIST(torch.utils.data.Dataset):
    def __init__(self, base_dataset, eta, target_label, seed=42):
        self.base = base_dataset
        n_total = len(base_dataset)
        n_poison = max(1, int(round(eta * n_total))) if eta > 0 else 0
        g = torch.Generator()
        g.manual_seed(seed)
        all_idx = torch.randperm(n_total, generator=g)
        self.poison_idx = set(all_idx[:n_poison].tolist())

```

```

        self.target_label = target_label
        self.n_poison = n_poison
        self.n_total = n_total

    def __len__(self):
        return len(self.base)

    def __getitem__(self, idx):
        img, label = self.base[idx]
        if idx in self.poison_idx:
            img = add_trigger(img)
            label = self.target_label
        return img, label

def train_backdoor_model(train_dataset, eta, epochs=5):
    poisoned = PoisonedMNIST(train_dataset, eta=eta, target_label=TARGET_LABEL)
    loader = torch.utils.data.DataLoader(poisoned, batch_size=128, shuffle=True)
    model = Net().to(DEVICE)
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    losses = []
    for _ in range(epochs):
        model.train()
        total_loss, n = 0.0, 0
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            optimizer.zero_grad()
            output = model(normalize(x))
            loss = F.nll_loss(output, y)
            loss.backward()
            optimizer.step()
            total_loss += loss.item() * y.size(0)
            n += y.size(0)
        losses.append(total_loss / n)
    return model, poisoned.n_poison, losses

def eval_clean(model, loader):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            pred = model(normalize(x)).argmax(dim=1)
            correct += (pred == y).sum().item()
            total += y.size(0)
    return correct / total

def eval_backdoor(model, loader, target_label):
    model.eval()
    success, total = 0, 0
    with torch.no_grad():

```

```

    for x, y in loader:
        x, y = x.to(DEVICE), y.to(DEVICE)
        mask = y != target_label
        x = x[mask]
        if x.numel() == 0:
            continue
        x_trig = add_trigger_batch(x)
        pred = model(normalize(x_trig)).argmax(dim=1)
        success += (pred == target_label).sum().item()
        total += x.size(0)
    return success / total if total > 0 else 0.0

```

```

def train_and_eval(train_dataset, test_loader, eta):
    torch.manual_seed(42)
    model, n_poison, losses = train_backdoor_model(train_dataset, eta)
    clean = eval_clean(model, test_loader)
    bd = eval_backdoor(model, test_loader, TARGET_LABEL)
    return model, n_poison, losses, clean, bd

```

```

def plot_trigger_demo(test_dataset, save_path):
    fig, axes = plt.subplots(2, 8, figsize=(16, 4.4))
    indices = [3, 6, 18, 25, 30, 50, 100, 200]
    for col, idx in enumerate(indices):
        img, label = test_dataset[idx]
        trig_img = add_trigger(img)
        axes[0, col].imshow(img.squeeze().numpy(), cmap="gray", vmin=0, vmax=1)
        axes[0, col].set_title(f"orig: {label}", fontsize=10)
        axes[0, col].axis("off")
        axes[1, col].imshow(trig_img.squeeze().numpy(), cmap="gray", vmin=0, vmax=1)
        axes[1, col].set_title("with trigger", fontsize=10, color="darkorange")
        axes[1, col].axis("off")
    fig.suptitle("Backdoor trigger: 3x3 '+' (灰度 0.6) 位于右下角", fontsize=12)
    plt.tight_layout()
    plt.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    print(f"[saved] {save_path}", flush=True)

```

```

def plot_backdoor_samples(model, test_dataset, save_path, n=8):
    model.eval()
    chosen = []
    for idx in range(len(test_dataset)):
        if len(chosen) >= n:
            break
        img, label = test_dataset[idx]
        if label == TARGET_LABEL:
            continue
        trig_img = add_trigger(img)
        with torch.no_grad():
            pred = model(normalize(trig_img.unsqueeze(0).to(DEVICE))).argmax(dim=1).item()
            pred_clean = model(normalize(img.unsqueeze(0).to(DEVICE))).argmax(dim=1).item()

```

```

        chosen.append((img.squeeze().numpy(), trig_img.squeeze().numpy(), label, pred_clean,
pred))

fig, axes = plt.subplots(2, n, figsize=(2 * n, 4.4))
for col, (orig, trig, label, p_clean, p_trig) in enumerate(chosen):
    axes[0, col].imshow(orig, cmap="gray", vmin=0, vmax=1)
    axes[0, col].set_title(f"true:{label}  clean→{p_clean}", fontsize=9)
    axes[0, col].axis("off")
    color = "red" if p_trig == TARGET_LABEL else "black"
    axes[1, col].imshow(trig, cmap="gray", vmin=0, vmax=1)
    axes[1, col].set_title(f"trigger→{p_trig}", fontsize=10, color=color)
    axes[1, col].axis("off")
fig.suptitle(f"Backdoor attack examples (target={TARGET_LABEL})", fontsize=12)
plt.tight_layout()
plt.savefig(save_path, dpi=150, bbox_inches="tight")
plt.close(fig)
print(f"[saved] {save_path}", flush=True)

def plot_eta_curve(eta_list, clean_list, bd_list, save_path):
    fig, ax = plt.subplots(figsize=(7.2, 4.4))
    ax.plot(eta_list, clean_list, "-o", color="#2ca02c", label="clean accuracy")
    ax.plot(eta_list, bd_list, "-s", color="#d62728", label="backdoor success rate")
    ax.axhline(THRESHOLD, color="gray", linestyle="--", label=f"target = {THRESHOLD:.0%}")
    ax.set_xlabel("η (poison ratio)")
    ax.set_ylabel("rate")
    ax.set_title("Backdoor: clean acc & backdoor success vs η")
    ax.set_ylim(0, 1.05)
    ax.grid(True, alpha=0.3)
    ax.legend()
    plt.tight_layout()
    plt.savefig(save_path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    print(f"[saved] {save_path}", flush=True)

def search_min_eta(train_dataset, test_loader, eta_candidates):
    """在候选 η 列表中找到后门成功率 >95% 的最小值。"""
    results = []
    min_eta = None
    min_model = None

    for eta in eta_candidates:
        print("\n" + "=" * 60, flush=True)
        print(f"训练 backdoor 模型 (η = {eta})", flush=True)
        print("=" * 60, flush=True)
        model, n_poison, losses, clean, bd = train_and_eval(train_dataset, test_loader, eta)
        print(f"共注入 {n_poison} / 60000 个 trigger 样本", flush=True)
        for i, loss in enumerate(losses, 1):
            print(f"epoch {i} | avg_loss = {loss:.4f}", flush=True)
        print(f">>> η={eta:.4f} | clean_acc = {clean * 100:.2f}% | "
              f"backdoor_success = {bd * 100:.2f}%", flush=True)
        results.append((eta, n_poison, clean, bd, losses[-1]))

```

```

        if bd > THRESHOLD and (min_eta is None or eta < min_eta):
            min_eta = eta
            min_model = model

    return results, min_eta, min_model

def main():
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    os.makedirs(IMAGE_DIR, exist_ok=True)
    os.makedirs(MODEL_DIR, exist_ok=True)
    log_path = os.path.join(OUTPUT_DIR, "task2_run.log")

    class Tee:
        def __init__(self, *files):
            self.files = files
        def write(self, data):
            for f in self.files:
                f.write(data)
                f.flush()
        def flush(self):
            for f in self.files:
                f.flush()

    with open(log_path, "w", encoding="utf-8") as log_file:
        old_stdout = sys.stdout
        sys.stdout = Tee(old_stdout, log_file)
        try:
            _run()
        finally:
            sys.stdout = old_stdout

def _run():
    torch.manual_seed(42)
    transform = transforms.Compose([transforms.ToTensor()])
    train_dataset = datasets.MNIST("./data", train=True, download=False,
transform=transform)
    test_dataset = datasets.MNIST("./data", train=False, download=False,
transform=transform)
    test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=256, shuffle=False)

    plot_trigger_demo(test_dataset, os.path.join(IMAGE_DIR, "task2_trigger_demo.png"))

    # 细粒度  $\eta$  搜索
    eta_candidates = [0.0005, 0.0008, 0.001, 0.002, 0.005, 0.01]
    results, min_eta, min_model = search_min_eta(train_dataset, test_loader, eta_candidates)

    # PDF 硬性要求  $\eta=0.01$  单独验证并保存模型
    print("\n" + "=" * 60, flush=True)
    print("PDF 硬性要求验证:  $\eta = 0.01$ ", flush=True)
    print("=" * 60, flush=True)
    torch.manual_seed(42)

```

```

model_001, n_poison, losses, clean_001, bd_001 = train_and_eval(train_dataset,
test_loader, 0.01)
print(f" 共注入 {n_poison} / 60000 个 trigger 样本", flush=True)
for i, loss in enumerate(losses, 1):
    print(f"  epoch {i} | avg_loss = {loss:.4f}", flush=True)
print(f"  >>>  $\eta$ =0.0100 | clean_acc = {clean_001 * 100:.2f}% | "
      f"backdoor_success = {bd_001 * 100:.2f}%", flush=True)

backdoor_path = os.path.join(MODEL_DIR, "mnist_cnn_backdoor.pth")
torch.save(model_001.state_dict(), backdoor_path)
print(f"[saved] {backdoor_path}", flush=True)

plot_backdoor_samples(model_001, test_dataset, os.path.join(IMAGE_DIR,
"task2_backdoor_samples.png"))

eta_list = [r[0] for r in results]
clean_list = [r[2] for r in results]
bd_list = [r[3] for r in results]
plot_eta_curve(eta_list, clean_list, bd_list, os.path.join(IMAGE_DIR,
"task2_eta_curve.png"))

result_path = os.path.join(OUTPUT_DIR, "task2_results.txt")
lines = [
    f"Backdoor target = {TARGET_LABEL}",
    f"Trigger: 3x3 '+' at {TRIGGER_CENTER}, value={TRIGGER_VALUE}",
    f"Threshold = {THRESHOLD:.0%}",
    "",
    "===  $\eta$  扫描结果 ===",
]
for eta, n_poison, clean, bd, final_loss in results:
    lines.append(
        f"eta={eta:.4f}\t"
        f"n_poison={n_poison}\t"
        f"clean_acc={clean*100:.2f}%\t"
        f"backdoor_success={bd*100:.2f}%\t"
        f"final_loss={final_loss:.4f}"
    )
lines += [
    "",
    f"=== 最小满足  $\eta$  (backdoor_success > {THRESHOLD:.0%}) ===",
    f"min_eta = {min_eta:.4f}" if min_eta else "min_eta = N/A",
    "",
    "=== PDF 硬性要求  $\eta$ =0.01 ===",
    f"clean_acc = {clean_001*100:.2f}%",
    f"backdoor_success = {bd_001*100:.2f}%",
]
with open(result_path, "w", encoding="utf-8") as f:
    f.write("\n".join(lines))
print(f"结果已写入 {result_path}", flush=True)

if __name__ == "__main__":
    main()

```

4.1 实验设置

参数	值
目标标签	1
Trigger	右下角 3×3 「+」 形，像素值 0.6，中心 (25,25)
投毒比例 η	扫描 + PDF 硬性验证 $\eta=0.01$
训练	5 epoch, Adam lr=0.001, batch=128
成功标准	带 trigger 样本被分为 1 的比例 > 95%

Trigger 注入逻辑：

```
def add_trigger(img):
    img = img.clone()
    r, c = TRIGGER_CENTER
    v = TRIGGER_VALUE
    img[0, r, c] = v
    img[0, r - 1, c] = v
    ...
    return img
```

投毒数据集按固定种子随机选取 $\eta \times 60000$ 个训练索引，替换标签为 1 并加 trigger。

4.2 实验结果

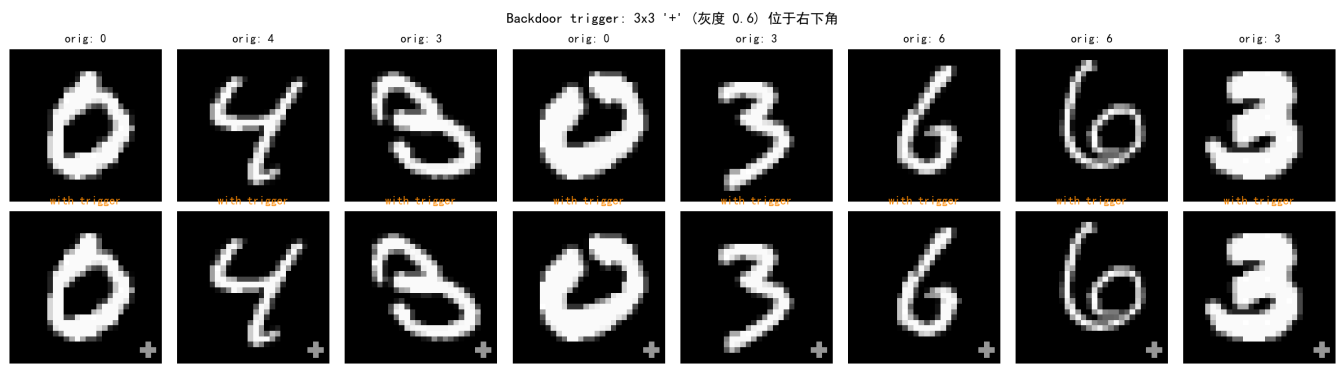
η	投毒样本数	干净准确率	后门成功率
0.0005	30	98.91%	76.14%
0.0008	48	99.01%	92.23%
0.0010	60	98.93%	98.91%
0.0020	120	98.75%	99.39%
0.0050	300	98.93%	99.95%
0.0100	600	98.76%	99.95%

- 满足 95% 后门成功率的最小 $\eta = 0.001$ （60 个投毒样本）。
- $\eta = 0.01$ 时后门成功率 **99.95%**，干净准确率仍约 **98.76%**，满足实验说明中的隐蔽性与有效性要求。

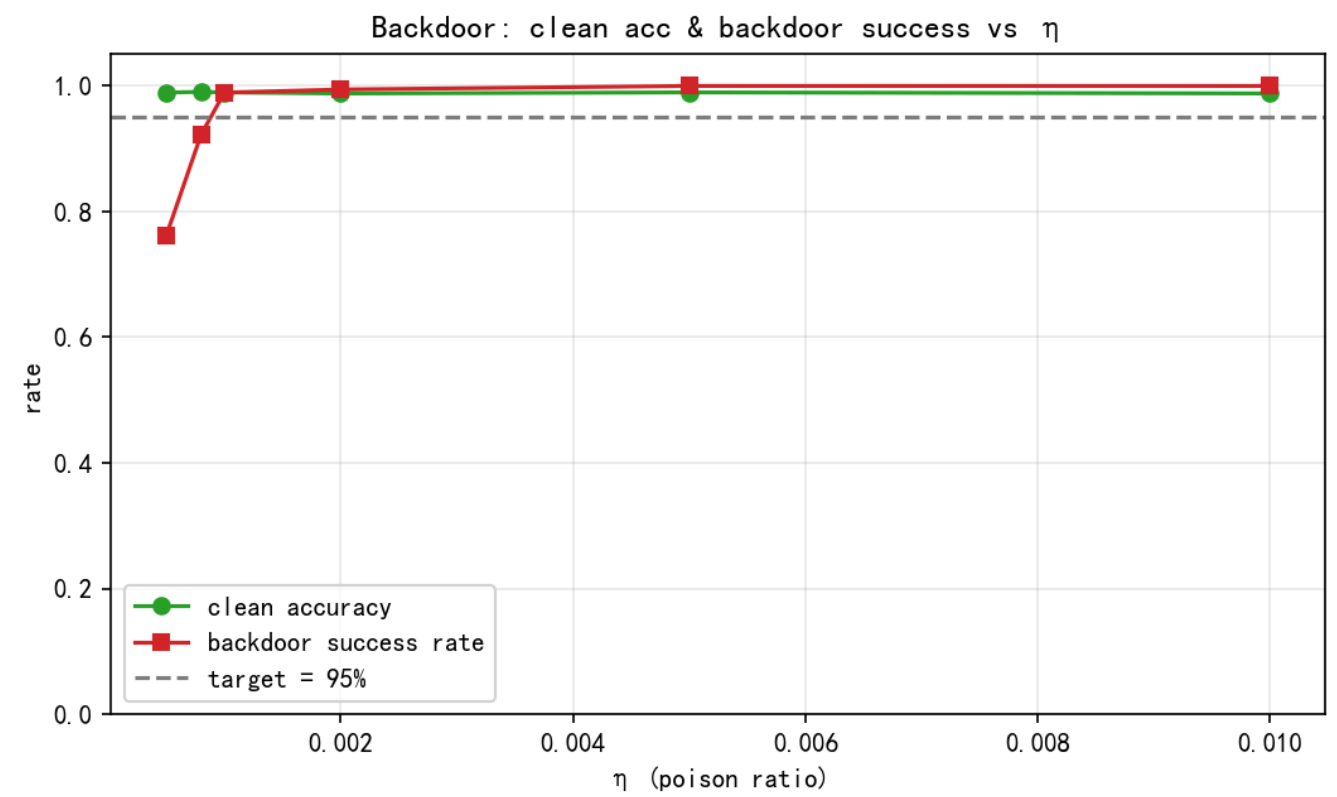
后门模型权重：`outputs/models/mnist_cnn_backdoor.pth`。

4.3 结果图表

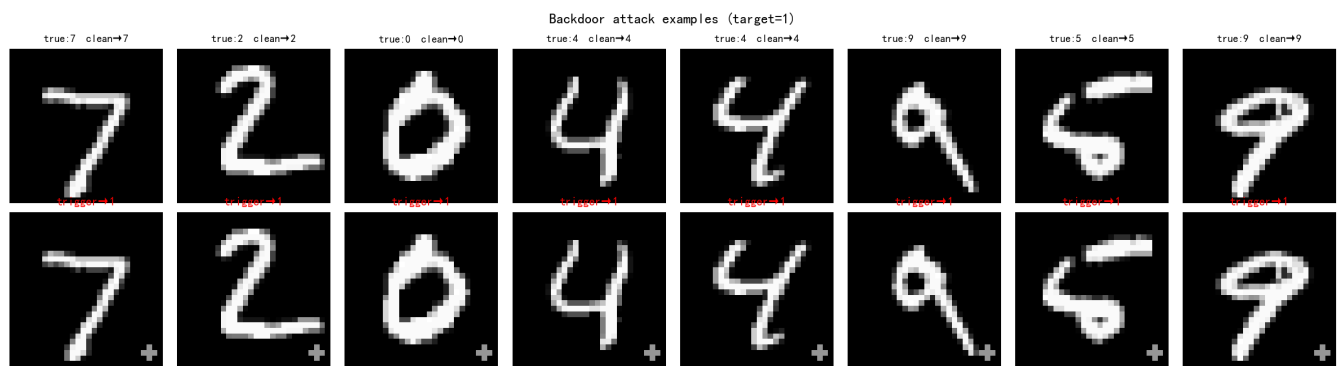
Trigger 示意



η 与准确率 / 后门成功率



后门攻击样例



上行：干净图像及预测；下行：加 trigger 后模型输出目标类 1（红字标注）。

5 复现实验

```
# 1. 训练干净模型（若尚无 mnist_cnn.pth）
python model.py

# 2. Task1（三种方法，耗时较长，建议 CPU 上耐心等待）
python task1_fgsm.py

# 仅跑某一种方法
python task1_fgsm.py --method pgd

# 3. Task2
python task2_backdoor.py
```

6 总结

1. 以 **95% 攻击成功率** 为统一基准，**PGD / PGD+** 可在 $\epsilon \approx 0.16 \sim 0.17$ (**无目标**) 达到要求，优于助教所述 PGD「约 0.2」、优化方法「约 0.15」的量级讨论；**标准 FGSM** 在本模型上无法达到 95%，但在 $\epsilon = 0.3$ 时无目标错误率约 **82%**，与「FGSM 可用 0.3 扰动预算」的经验一致。
2. **有目标攻击** 需要更大 ϵ (PGD 0.30, PGD+ 0.28)，迭代攻击仍是必选方案。
3. **后门攻击** 在 $\eta = 0.001$ 即可超过 95% 成功率；PDF 规定的 $\eta = 0.01$ 后门成功率接近 100%，且干净精度几乎不变，说明 trigger 隐蔽而有效。